

AL-2002

Artificial Intelligent Lab

Final Project

Note: Carefully read the following instructions (Each instruction contains a weightage)

- **3 students** are allowed per group.
- Your code should be clear and add exceptions where required [to avoid system crashing or accept incorrect data/options]
- Project is to be submitted as a zip file.
- Submit the whole project by one member of the group.
- There must be a block of comments at the start of each function; the block should contain brief description about functionality of code.
- Use understandable name of variables.
- Proper indentation of code is essential.
- Make a report file on Microsoft Word and paste all your code with all possible screenshots of every task [also the information about modules completed and the sequence to run the program].
- At the end when you are done with your tasks, make your submission on Google classroom. (Make sure your submission is completed).
- Please submit your file in this format 23F-1234_23F-1235_23F-1235_Project.zip.

Smart City Traffic & Emergency Response AI System

Project Introduction

Urban traffic systems are complex, dynamic, and highly time-sensitive. A modern city must support efficient movement for ordinary vehicles while ensuring that emergency services such as ambulances, fire units, and police vehicles can operate effectively under congested conditions. At the same time, traffic control must remain safe, policy-compliant, and responsive to changing road situations.

The **Smart City Traffic & Emergency Response AI System** addresses this challenge by modeling traffic management as an integrated AI problem. Rather than relying on a single technique, the system uses multiple AI modules that are activated based on the type of request. Routine traffic requests may require only route generation, while emergency situations may involve priority prediction, policy validation, traffic signal control, and optimized navigation working together.

The project focuses on four key functional areas: traffic routing, emergency vehicle movement, traffic signal control, and traffic density analysis. These components operate within a unified architecture, enabling coordinated decision-making instead of isolated recommendations.

At a high level, the system processes a structured traffic request through validation and normalization, selects the appropriate processing pipeline based on request type, executes the required AI modules in sequence, and generates a final response. This response may include a recommended route, predicted priority level, control decision, validation result, or a combined action plan.

Project Scope and Functional Focus

The system is focused on operational city-traffic decision support. Its scope includes the intelligent handling of traffic requests that are relevant to road movement, emergency mobility, and signal-level control decisions.

- Traffic routing for normal road users
- Emergency vehicle prioritization and movement support
- Traffic signal coordination under timing and safety constraints
- Traffic density-informed decision support
- Integrated response generation for both civilian and emergency scenarios

Project Objectives

The objectives of the Smart City Traffic & Emergency Response AI System are listed below. These objectives define what the project is intended to demonstrate both technically and conceptually.

- To design an AI-based traffic management platform for a smart city environment.
- To support route generation for both regular and emergency movement requests.
- To prioritize emergency traffic using predictive urgency analysis.
- To validate traffic-control actions using policy and rule-based reasoning.
- To assign or adjust signal-control actions under constraints.
- To combine multiple AI methods inside one modular, explainable system.
- To produce clear and actionable final responses for system users or city operators.

System Users and Operational Roles

The system may be used directly or indirectly by different operational roles within a smart city environment. These roles affect both request semantics and system response requirements.

Role	Typical Request	Operational Need
Civilian Driver	Route recommendation to a destination	Efficient navigation under normal traffic conditions
Emergency Vehicle Unit	Priority route and rapid corridor movement	Reduced response time and operational priority
Traffic Control Operator	Signal adjustment or traffic-control intervention	Safe and coordinated traffic management
Emergency Coordination Center	Combined validation, priority, and route support	Fast, authorized, and explainable response planning

Request Categories

To preserve control-flow clarity, the system processes traffic requests through fixed request categories. Each category corresponds to a specific type of operational need and activates only the modules required for that need.

Request Category	Purpose	Typical Modules	Example
Route_Request	Standard route guidance for normal traffic movement	Search	Best route from junction _A to District _B
Policy_Check	Validation of rule-based traffic or access conditions	Logic / Knowledge Base	Is signal override allowed in this zone?
Control_Allocation_Request	Signal or lane-control assignment under constraints	Logic / Knowledge Base → CSP	Assign a safe signal sequence for a controlled corridor
Emergency_Response_Request	Priority estimation, policy validation, and emergency routing support	ANN → Logic / Knowledge Base → CSP → Search	Ambulance routing through congested sector
Integrated_City_Service_Request	Combined routing, control. And response support	ANN → Logic / Knowledge Base → CSP → Search → Final Response	Emergency movement with corridor control and guided route execution

System Architecture Overview

The architecture of the system is pipeline-based and modular. Each request follows a structured progression from input to final output. The system does not use every module for every request. Instead, the request category determines the processing path.

Traffic Request → Input & Preprocessing → Request Router → AI Modules → Final Response

Core Data Requirements

Because the system is structured and operational, it depends on a defined set of input data fields. These fields support routing, policy checking, urgency prediction, and control allocation.

Common input fields may include:

- request_id
- vehicle_type

- request_category
- current_location
- destination
- incident_severity
- time_sensitivity
- traffic_density
- priority_claim
- control_zone
- description_note

Typical derived or internal fields may include estimated distance, route cost, policy status, assigned control action, and predicted priority level. Structured data is preferred because it supports deterministic validation and clean module interaction.

Major Modules Overview

The project contains seven major modules. Each module performs a distinct function inside the system, and together these modules form the complete operational workflow.

1. Input & Preprocessing Module

This module is the entry point of the system. Its role is to receive a traffic request, validate the submitted information, normalize field values, and build a standard internal request object. It ensures that road identifiers, control-zone labels, and request categories follow predefined system conventions.

For requests that need machine learning-based priority prediction, this module also prepares the ANN feature vector. In the smart traffic scenario, relevant features may include vehicle type, request type, severity, time sensitivity, traffic density, and estimated route distance. This module is therefore both a validation layer and a preparation layer.

All inputs to the system are assumed to be manually entered in structured form. The system does not process free-text commands and does not use Natural Language Processing (NLP) techniques.

Input preprocessing is limited to field validation, normalization, and categorical mapping. The objective is to ensure clean and consistent inputs for logic reasoning, search, and ANN processing rather than language understanding or data extraction.

2. Request Router

The Request Router is the control-flow manager of the system. It receives the standardized request and determines which processing path should be followed. A normal route request may only need Search. A policy check may only need Logic / Knowledge Base. An emergency request may require ANN, policy validation, control allocation, and routing together.

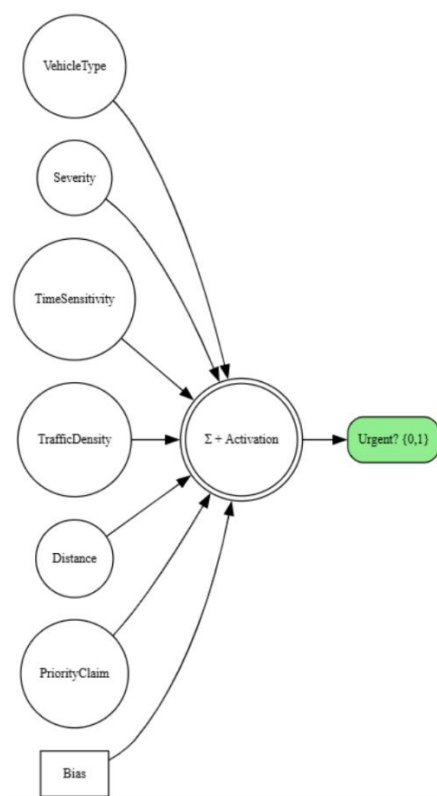
The router is important because it keeps the system disciplined. It prevents inappropriate module calls, preserves sequencing rules, and ensures that each request is handled according to operational need rather than by a one-size-fits-all pipeline.

3. ANN Priority Module

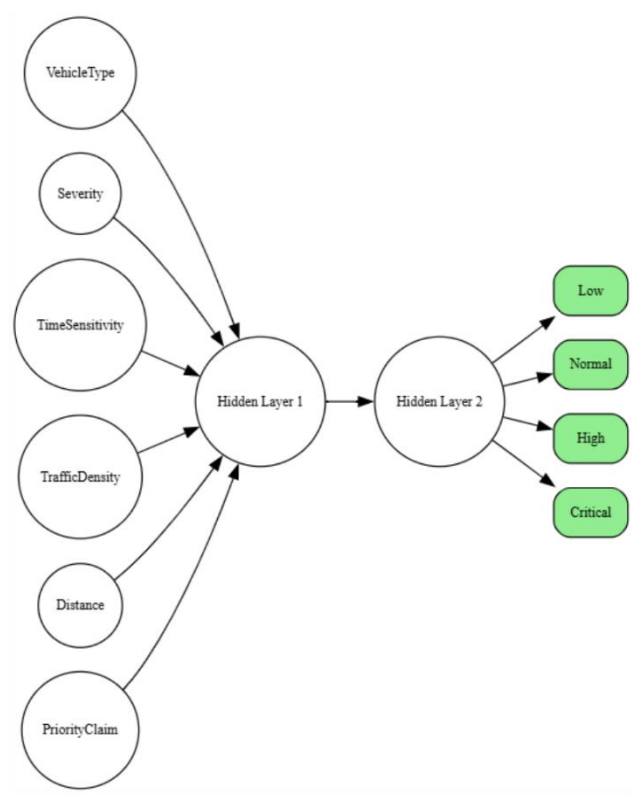
The ANN module estimates urgency or priority for requests that require priority-aware handling. In the traffic domain, this is especially important for emergency response. The module is not designed to decide whether a request is legally allowed. Instead, it predicts how urgent a situation appears based on structured operational indicators.

A binary baseline classifier may separate urgent from non-urgent requests, while a multiclass model may classify requests into levels such as low, normal, high, and critical. This information can later influence control allocation and final response messaging.

Binary Classifier



MLP Diagram



4. Logic / Knowledge Base Module

This module is responsible for policy validation and rule-based reasoning. It checks whether an action is allowed under traffic policy, whether emergency priority can be granted, whether

signal override conditions are satisfied, and whether a requested control action is logically consistent with the knowledge base.

In practical terms, this module protects the system from unsafe or unauthorized actions. It ensures that emergency requests are validated before the system adjusts signals or grants corridor priority. It is therefore the gatekeeper before any constrained control allocation is executed.

Required Predicates and Rules:

Vehicle(v)

EmergencyVehicle(v)

CivilianVehicle(v)

Location(l)

SignalZone(z)

Hospital(h)

Request(req)

RequestType(req,type)

CurrentLocation(v,l)

Destination(v,l)

IncidentSeverity(v,level)

TimeSensitive(v)

Priority(v,level)

Authorized(v,action)

AllowedAction(v,action)

EmergencyCorridor(v)

EmergencyRoute(v)

SignalOverride(z)

Approved(v,req)

Rejected(v, req)

$\text{EmergencyVehicle}(v) \wedge \text{IncidentSeverity}(v, \text{High}) \rightarrow \text{Priority}(v, \text{Critical})$

$\text{EmergencyVehicle}(v) \wedge \text{TimeSensitive}(v) \rightarrow \text{Priority}(v, \text{High})$

$\text{CivilianVehicle}(v) \rightarrow \text{Priority}(v, \text{Normal})$

$\text{EmergencyVehicle}(v) \wedge \text{SignalZone}(z) \rightarrow \text{Authorized}(v, \text{SignalOverride}(z))$

$\text{CivilianVehicle}(v) \wedge \text{SignalZone}(z) \rightarrow \neg \text{Authorized}(v, \text{SignalOverride}(z))$

$\text{EmergencyVehicle}(v) \wedge \text{Destination}(v, h) \wedge \text{Hospital}(h) \rightarrow \text{EmergencyCorridor}(v)$
 $\text{EmergencyCorridor}(v) \rightarrow \text{Authorized}(v, \text{EmergencyRoute})$

$\text{Authorized}(v, \text{action}) \rightarrow \text{AllowedAction}(v, \text{action})$
 $\neg \text{AllowedAction}(v, \text{action}) \rightarrow \text{Rejected}(v, \text{req})$

$\text{Priority}(v, \text{Critical}) \wedge \neg \text{Authorized}(v, \text{action}) \rightarrow \neg \text{AllowedAction}(v, \text{action})$
 $\text{Priority}(v, \text{Critical}) \wedge \text{Authorized}(v, \text{EmergencyRoute}) \rightarrow \text{AllowedAction}(v, \text{SignalOverride})$

$\text{AllowedAction}(v, \text{action}) \rightarrow \text{Approved}(v, \text{req})$

$\text{RequestType}(\text{req}, \text{Route_Request}) \rightarrow \text{Approved}(v, \text{req})$
 $\text{RequestType}(\text{req}, \text{Policy_Check}) \wedge \text{Authorized}(v, \text{action}) \rightarrow \text{Approved}(v, \text{req})$
 $\text{RequestType}(\text{req}, \text{Policy_Check}) \wedge \neg \text{Authorized}(v, \text{action}) \rightarrow \text{Rejected}(v, \text{req})$

$\text{RequestType}(\text{req}, \text{Control_Allocation_Request}) \wedge \text{AllowedAction}(v, \text{action}) \rightarrow \text{Approved}(v, \text{req})$
 $\text{RequestType}(\text{req}, \text{Control_Allocation_Request}) \wedge \neg \text{AllowedAction}(v, \text{action}) \rightarrow \text{Rejected}(v, \text{req})$

$\text{RequestType}(\text{req}, \text{Emergency_Response_Request}) \wedge \text{Priority}(v, \text{level}) \wedge$
 $\text{Authorized}(v, \text{EmergencyRoute}) \rightarrow \text{Approved}(v, \text{req})$

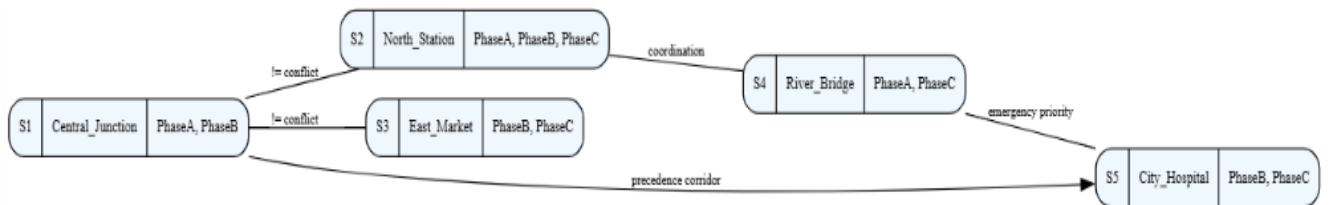
$\text{RequestType}(\text{req}, \text{Integrated_City_Service_Request}) \wedge \text{Priority}(v, \text{Critical}) \wedge$
 $\text{Authorized}(v, \text{EmergencyRoute}) \wedge \text{AllowedAction}(v, \text{action}) \rightarrow \text{Approved}(v, \text{req})$

5. CSP Scheduler / Control Allocation Module

The CSP module handles constrained control decisions. In the smart city traffic scenario, these decisions may involve signal timing plans, lane-control adjustments, emergency corridor allocation, or intersection sequence coordination. The module searches for a valid assignment that respects safety, timing, and conflict constraints.

This module is especially important when multiple intersections or control points must be coordinated together. The objective is not arbitrary control adjustment; the objective is a control plan that satisfies operational constraints while supporting the current traffic need.

CSP Graph

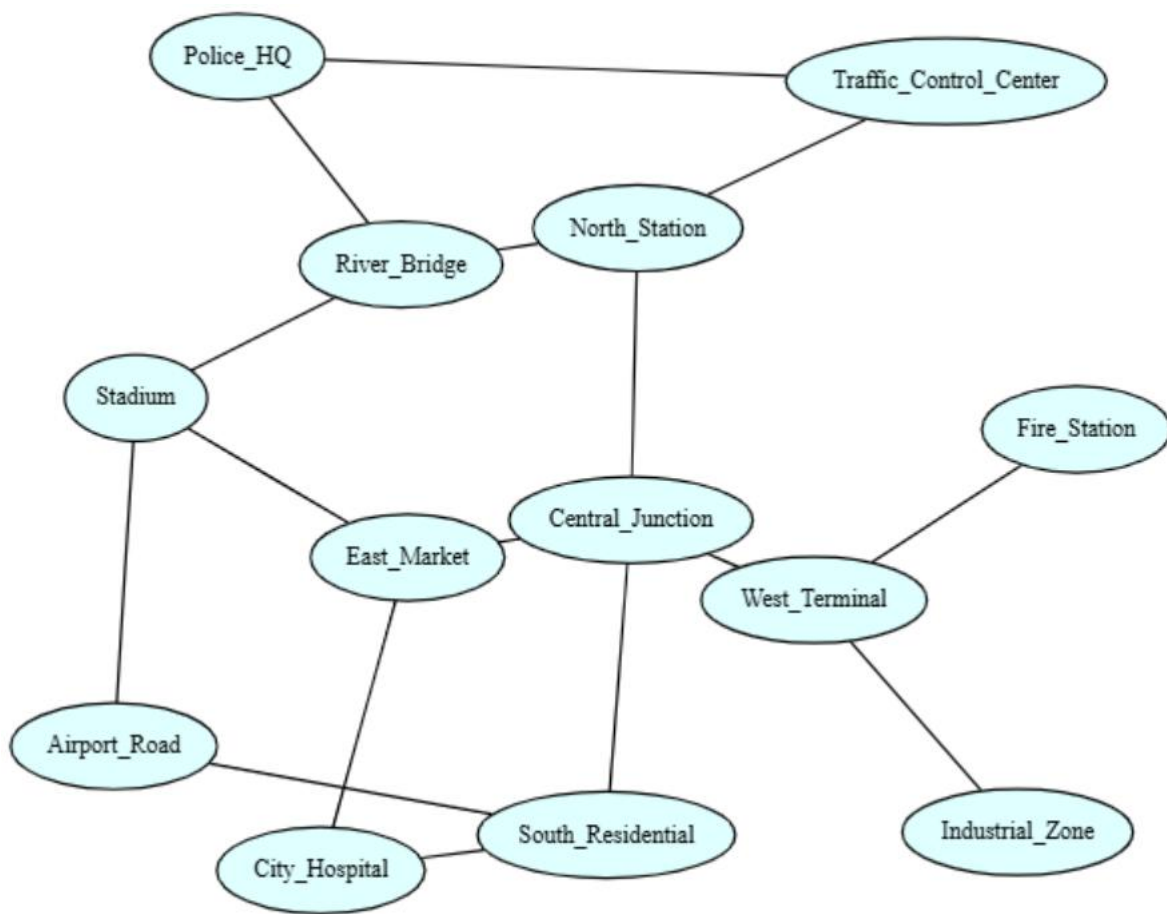


6. Search & Navigation Module

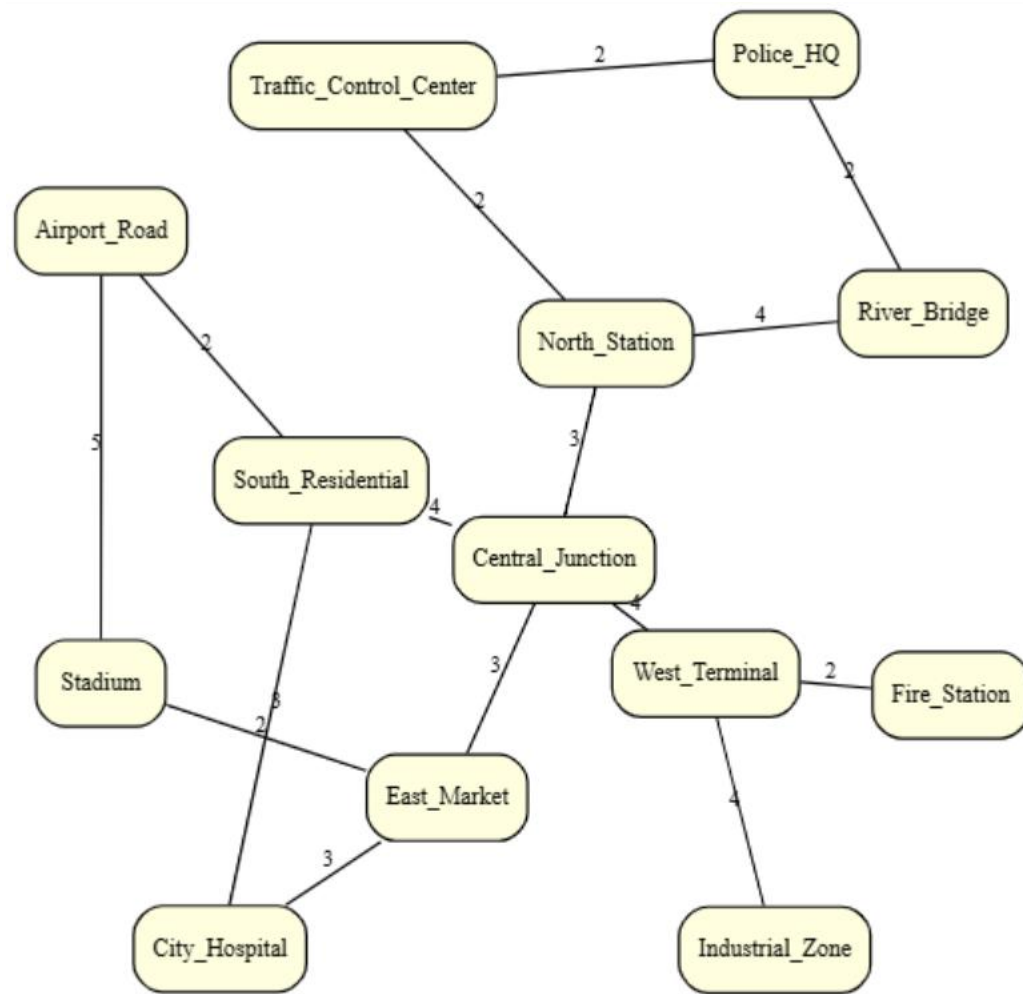
The Search module performs route generation over the city road network. Intersections may be modeled as nodes and roads as edges, with optional weights representing time, congestion, delay, or access penalties. This allows the system to support both standard and emergency route computation.

Operationally, the system may use BFS for unweighted route problems, A* for weighted problems when heuristic guidance is available, and UCS as a fallback for weighted problems without a suitable heuristic. Search may operate directly on a route request, or it may run after control allocation when the final route depends on approved traffic-control actions.

City Unweighted Graph



City Weighted Graph



7. Final Response Layer

The Final Response layer aggregates the outputs of the modules that were actually used for the request. It may present a selected route, a validated status, a predicted priority level, an assigned control plan, or an integrated response message. This layer is essential because operational systems must not only make decisions; they must also communicate them clearly.

Standard System Workflow

A typical request passes through the following workflow:

1. The traffic request is submitted with structured data.
2. Input & Preprocessing validates and standardizes the request.
3. The Request Router selects the required pipeline.
4. If needed, the ANN module predicts urgency or priority.

5. If needed, the Logic / Knowledge Base module validates policy and authorization conditions.
6. If needed, the CSP module assigns signal or control actions under constraints.
7. If needed, the Search module computes the final route.
8. The Final Response layer combines the outputs and presents the decision.

Final Response Design

The final response should be selective, operationally meaningful, and explainable. It should not include information from modules that were not used for the current request. For example, a standard route request should not display an ANN priority field if no urgency prediction was needed.

Depending on the request, the final response may include:

- Recommended route
- Estimated travel or delay information
- Predicted priority level
- Policy validation result
- Assigned signal-control or lane-control action
- Decision message and explanatory text

A typical emergency-response result may communicate that emergency priority has been validated, a control corridor has been allocated, and a fastest route has been generated. A typical route-only result may simply show the selected path and travel estimate.

End-to-End Example Scenario

Consider a request submitted for an ambulance moving from Central_Junction to City_Hospital during a high-density traffic period. The request includes the current location, destination, emergency designation, incident severity, and time sensitivity level.

First, Input & Preprocessing validates the request and prepares the structured feature vector. Next, the Request Router classifies the request as an emergency-response case that requires ANN, policy validation, control allocation, and navigation.

The ANN module predicts a critical priority level because the request involves emergency movement, severe conditions, and high time sensitivity. The Logic / Knowledge Base module then verifies that emergency-priority movement is authorized and that corridor-support actions are allowed. Once validated, the CSP module assigns a feasible sequence of signal-control decisions across the necessary intersections. After the control actions are assigned, the Search module computes the best route under the approved conditions. Finally, the response layer presents a coordinated output: critical priority, approved emergency path, assigned signal support, and selected route to City_Hospital.

Design Assumptions and Constraints

The project depends on a number of working assumptions that should be stated clearly in documentation.

- Traffic requests are submitted in a structured form.
- The city road network is represented using predefined nodes and edges.
- Traffic policy rules are internally consistent.
- Emergency claims are subject to policy validation.
- Priority prediction assists decision support but does not replace authorization logic.
- Control allocation respects hard safety constraints.

Implementation Guidelines and Scope Clarification

- The project is a software-based simulation system developed for academic purposes only. Real-world traffic sensors, live data feeds, or physical deployment are not required.
- All system inputs are manually defined and entered in a structured format (e.g., dictionaries, objects, or fixed input forms).
- The city map may be modeled using a small, hypothetical graph containing a limited number of intersections and roads sufficient to demonstrate search and control logic.
- Search algorithms (BFS, UCS, A*) should operate on this predefined graph and are not expected to handle real city-scale data.
- The Logic / Knowledge Base module may be implemented using rule-based conditions, if-else logic, or symbolic representations to simulate reasoning.
- The CSP Scheduler may assume a limited number of constraints and control points (e.g., 2–5 intersections) to demonstrate constraint satisfaction.
- The ANN module is intended to demonstrate priority estimation behavior only and may be implemented using simplified architecture and manually prepared examples.

Evaluation Rubrics

Component	Marks
Input & Preprocessing Module	10
ANN Priority Prediction Module	20
Logic / Knowledge Base Module	20
CSP Scheduler / Control Allocation	15
Search Navigation Module	15
Final Response & Integration	10
Documentation & Code Quality	10
Total	100

Submission Guidelines

Students are required to submit:

1. Complete Source Code (ZIP format)
2. Project Report (PDF Format):
 - Project Overview
 - Description of AI Concepts Used
 - Code Snippets
 - Screenshots

BEST OF LUCK!